

OMI (Orchestrated Math Interpreter)

5단계 심층 추론 및 역공학 데이터셋 기반의 AI 수학 튜터 시스템

AIMO Project Portfolio

May 17, 2026

Abstract

대형 언어 모델(LLM)은 뛰어난 언어 능력을 갖추었으나, 복잡한 수학 문제 앞에서는 자주 거짓말(환각, Hallucination)을 하거나 단순한 사칙연산조차 틀리는 치명적인 한계를 보입니다. 본 연구는 이러한 한계를 극복하기 위해 AI가 스스로 “어떻게 풀 것인지 계획을 세우고, 파이썬 코드를 작성해 계산을 맡기며, 스스로 답을 검증하는” 인간 수학자의 사고 과정을 그대로 모방한 **OMI (Orchestrated Math Interpreter)** 시스템을 제안합니다.

본 보고서에서는 AI가 5단계에 걸쳐 문제를 풀어내는 과정(5-Stage Deep Reasoning Pipeline), 여러 AI가 토론하며 정답을 찾아내는 앙상블 기법(Voting & Debate), 그리고 완벽히 깨끗한 학습 데이터를 만들어내는 역공학(Reverse Engineering) 기법을 소개합니다. 아울러 이 복잡한 과정을 일반 사용자도 직관적으로 이해할 수 있도록 구현한 ‘투명한 유리 상자(Glass-box)’ 웹 대시보드 구조에 대해 상세히 설명합니다.

Contents

1 Introduction (서론)

왜 AI 수학 튜터(OMI)를 만들었는가?

최근 챗GPT를 비롯한 AI는 글쓰기와 코딩에서 놀라운 성능을 보입니다. 하지만 “100! (팩토리얼)의 끝에 0이 몇 개야?”처럼 깊은 논리와 정확한 수식 계산이 필요한 질문을 던지면, 그럴듯한 거짓 말을 만들어 내는 경우가 많습니다. AI는 기본적으로 ‘다음 단어를 예측’하는 텍스트 생성기이기 때문입니다.

우리는 “AI에게 머릿속으로만 계산하게 하지 말고, 종이와 계산기(Python)를 쥐여주면 어떨까?”라는 질문에서 출발했습니다. 복잡한 문제를 단계별로 쪼개고, 계산은 파이썬 프로그램에게 맡기며, 나온 답을 여러 AI 에이전트가 검증하게 만드는 총체적 지휘 시스템, 즉 **오케스트레이터(Orchestrator)**를 만드는 것이 본 프로젝트의 핵심 목적입니다.

2 Related Work: AI는 왜 수학을 못할까? (관련 연구 및 배경)

AI의 수학적 한계를 보완하기 위해 학계에서 연구된 세 가지 핵심 개념을 우리의 방식으로 이해하기 쉽게 풀어보겠습니다.

1. TIR (Tool-Integrated Reasoning, 도구 결합 추론)

- **문제점:** AI는 계산기가 아닙니다. “ 12345×67890 ”을 머릿속으로만 계산하려고 하니 당연히 틀립니다.
- **해결책:** AI에게 **계산기(Python 코드) 사용법**을 알려줍니다. AI가 직접 계산하는 대신, “이 문제를 푸는 파이썬 코드를 짜 줘”라고 명령하고, 그 코드를 실행시킨 결과값을 가져오게 하는 기술입니다.

2. Pass@k 와 Majority Voting (다수결 투표)

- **문제점:** AI에게 어려운 문제를 한 번만 풀게 하면(Zero-shot), 실수할 확률이 너무 높습니다.
- **해결책:** 똑같은 문제를 AI에게 100번 풀게 합니다. 수학 문제의 특징상, 계산 실수를 하면 100번 모두 각각 다른 오답(발산)을 냅니다. 하지만 논리가 맞았다면 동일한 정답으로 몰리게 됩니다(수렴). 가장 많이 나온 답을 고르는 다수결 원칙이 바로 앙상블의 핵심입니다.

3. Multi-Agent Debate (다중 에이전트 토론)

- **문제점:** AI는 자기가 내뱉은 첫 번째 문장에 묶여버리는 **확증 편향(Confirmation Bias)**이 심합니다. 첫 단추를 잘못 끼우면 끝까지 억지를 부리며 오답을 정당화합니다.
- **해결책:** 역할을 나눕니다. 한 AI(Coder)는 코드만 짜고, **백지상태의 다른 AI(Reviewer)**가 그 코드의 **논리적 허점을 찾아내게** 합니다. 사람도 남의 오타를 훨씬 잘 찾는 것처럼, AI도 생성보다 검증에 훨씬 뛰어납니다.

3 Proposed Methodology (제안하는 방법론: 왜 이렇게 설계했는가?)

본 프로젝트의 심장인 5단계 파이프라인(5-Stage Deep Reasoning Pipeline)은 학생이 어려운 문제를 만났을 때 사고하는 과정을 그대로 AI 시스템으로 옮긴 것입니다.

3.1 5-Stage Deep Reasoning Pipeline의 구체적 원리

• Stage 1: Labeling (분석 및 분류)

- **왜 하는가?** 요리를 시작하기 전에 재료가 한식인지 양식인지 파악해야 알맞은 조리도구를 꺼낼 수 있습니다.

- **어떻게 하는가?** 문제가 정수론인지, 기하학인지 파악하고 핵심 숫자(예: $N = 100$)를 뽑아냅니다.

- **Stage 2: Retrieval (지식 로드)**

- **왜 하는가?** 백과사전 전체를 한 번에 주면 AI가 헛갈려합니다. 기하학 문제라면 기하학 공식만 꺼내 주어야 집중력(Context)이 올라갑니다.
- **어떻게 하는가?** 앞서 분류된 도메인에 맞춰 파이썬 라이브러리(sympy)나 수학적 정리(르장드르 공식 등)만 선택적으로 AI의 뇌에 주입합니다.

- **Stage 3: The Calculation Router (전략 라우터 - OMI의 꽃)**

- **왜 하는가?** 문제의 스케일에 따라 푸는 방식이 달라져야 하기 때문입니다.
- **어떻게 하는가?**
 - * **Path A (Simulator):** 숫자가 작거나 노가다성 탐색이 필요하면 직접 1부터 100까지 반복문을 도는 시뮬레이터 방식을 택합니다.
 - * **Path B (Theoretician):** 숫자가 너무 크면(예: 10^{100}) 직접 도는 것은 불가능합니다. 이때는 대수적 공식이나 정리를 이용해 0.1초 만에 푸는 이론적 방식을 택합니다.

- **Stage 4: Execution & Error Correction (실행 및 자가 수정)**

- **왜 하는가?** 짠 코드가 한 번에 완벽히 돌아갈 리 없습니다. 에러가 나면 포기하지 않고 고치는 과정이 필요합니다.
- **어떻게 하는가?** 안전한 샌드박스에서 코드를 실행합니다. 만약 `TimeoutError`(시간 초과)가 난다면, 모델은 “아, 숫자가 너무 커서 시뮬레이션은 무리구나. 이론적 방식으로 다시 짜야겠다”라고 스스로 피드백을 받고 전략을 수정합니다.

- **Stage 5: The Verification Router (최종 검증)**

- **왜 하는가?** 정답을 도출했더라도 마지막으로 한 번 더 의심해 보아야 치명적인 오답을 거를 수 있습니다.
- **어떻게 하는가?** **Scale-Down Test(축소 검증)**를 씁니다. 10^{100} 단위의 문제 로직을 완성했다면, 그 로직에 숫자 3을 넣어보고 손으로 푼 결과와 일치하는지 확인하는 식입니다. 이상이 없으면 최종 답안으로 확정합니다.

3.2 역공학(Reverse Engineering) 기반 합성 데이터 엔진

- **문제의식:** AI를 똑똑하게 훈련시키려면 좋은 수학 문제집(데이터셋)이 필요합니다. 그런데 인터넷에서 긁어오면 이미 AI가 학습한 문제라 변별력이 없고(오염), 사람이 직접 만들면 돈과 시간이 너무 듭니다.
- **해결책 (Open-Reverse-Math):** “문제를 내고 정답을 맞춰라”가 아니라, “정답을 42로 정해놓고, 그 답이 나올 수밖에 없는 복잡한 문제를 만들어라”라는 역발상을 적용했습니다.
- **왜 좋은가?** 답이 무조건 참(True)이라는 것이 보장되며, 세상에 존재하지 않는 완전히 새로운 문제(Novelty)를 수십만 개 무한정 생성할 수 있습니다. AI가 이 촌충한 ‘사고의 지도’를 따라 학습하게 되면, 답만 찍는 모델이 아니라 논리적 메타인지를 갖춘 튜터 모델로 성장합니다.

4 System Implementation (시스템 구현 및 대시보드 시각화)

백엔드에서 아무리 훌륭한 로직이 돌아가도, 사용자가 이를 볼 수 없다면 신뢰할 수 없습니다. 따라서 우리는 **AI의 사고 과정을 투명한 유리 상자(Glass-box)처럼 들여다볼 수 있는 교육용 대시보드**를 만들었습니다.

4.1 XAI(설명 가능한 AI)와 대시보드 구동 시나리오

단순히 “정답은 24입니다”라고 뱉는 AI는 교육용으로 부적합합니다. “왜 24인지, 왜 그 방식을 썼는지” 설명(XAI)해야 합니다.

[대시보드 실전 시나리오: 100! 의 끝에 0이 몇 개야?]

1. **분석 (Labeling):** 대시보드에 태그 아이콘이 뜨며 “이 문제는 정수론입니다. 핵심 숫자는 100입니다”라고 분석 결과를 화면에 띄워줍니다.
2. **지식 로드 (Retrieval):** 책 모양 아이콘과 함께 “팩토리얼의 0 개수를 구하기 위해 르장드르 정리를 로드했습니다”라는 팝업이 나타납니다.
3. **경로 선택 (Router):** 화면에 갈래길이 나타나고, AI가 “100은 직접 곱하기엔 너무 크므로 이론적 공식(Theoretician) 경로로 진입합니다”라며 경로에 불빛이 들어옵니다.
4. **코드 작성 및 실행 (Execution):** 화면 중앙의 가상 터미널 창에 AI가 스스로 타이핑하는 파이썬 코드가 주르륵 나타납니다. Run 버튼이 돌아가며 24라는 숫자를 뽑아냅니다.
5. **검증 (Verification):** 방패 아이콘과 함께 “로직이 맞는지 숫자 10을 넣어 테스트 중... 10! 의 0 개수인 2개와 일치합니다. 검증 완료!”라는 애니메이션이 뜹니다.
6. **최종 해설 (XAI):** “무식하게 100번을 곱하지 않고, 5의 배수를 찾는 코드를 짜서 0.01초 만에 풀었습니다.”라는 친절한 해설을 정답과 함께 보여줍니다.

5 Conclusion (결론)

지금까지의 AI가 단순히 ‘말을 잘하는 문과생’이었다면, OMI 프로젝트는 AI에게 ‘계산기 사용법’과 ‘검토하는 습관’을 길러주어 ‘**엄밀한 이과생**’으로 진화시키는 작업이었습니다.

본 프로젝트는 5단계 파이프라인, 다중 에이전트 토론, 파이썬 샌드박스 통합, 역공학 데이터셋이라는 최신 기술의 집약체입니다. 하지만 가장 큰 성과는 이 모든 복잡한 기술을 대시보드 위에 투명하게 올려놓아, 기술을 모르는 사람도 AI가 어떻게 생각하고 정답에 도달하는지 납득할 수 있게 문턱을 낮췄다는 데 있습니다. 이 아키텍처는 향후 코딩, 데이터 분석, 법률 논리 증명 등 모든 고도의 추론 분야에 활용될 수 있는 강력한 뼈대가 될 것입니다.

6 Appendix: 친절한 용어 사전 (Glossary)

이 보고서를 읽는 데 도움이 될 핵심 용어들을 최대한 일상적인 비유로 쉽게 풀어 설명합니다.

• 오케스트레이션 (Orchestration)

- 비유: 오케스트라의 지휘자.
- 설명: AI 에이전트 혼자 모든 걸 하는 게 아니라, 분석가 AI, 코더 AI, 검증가 AI 등 여러 전문가를 배치하고 “지금은 네가 나서고, 다음엔 네가 코드 짜”라며 지휘하는 전체 시스템 관리 기술을 말합니다.

• TIR (Tool-Integrated Reasoning, 도구 결합 추론)

- 비유: 머릿속 암산 대신 종이와 계산기 쓰기.
- 설명: AI가 수학 문제를 텍스트로만 푸는 것이 아니라, 파이썬 코드나 SymPy 같은 계산 도구를 실행해 그 결과를 바탕으로 추론하는 방법입니다.

• 다수결 투표 (Majority Voting) 와 Pass@k

- 비유: 백지장도 백 번 맞들면 낫다.

- **설명:** 모델이 조금 멍청하더라도 똑같은 문제를 각기 다른 방법으로 100번 풀게 시킵니다. 틀린 답은 엉뚱한 숫자로 뿔뿔이 흩어지지만, 논리가 맞았다면 하나의 정답으로 모이기 때문에 가장 많이 나온 답을 고르는 필승 전략입니다.

- **Scale-Down Test (축소 검증)**

- **비유:** 거대한 다리를 짓기 전에 미니어치 모델로 튼튼한지 테스트해 보기.
- **설명:** 10^{100} 같은 거대한 숫자가 들어간 로직을 짰다면, 3이나 10처럼 아주 작은 숫자를 대입해 보고 직접 손계산한 것과 일치하는지 확인해 AI의 로직을 검증하는 단계입니다.

- **역공학 데이터 생성 (Reverse Engineering Data Generation)**

- **비유:** 미로 찾기 책을 만들 때 입구에서 출발하는 게 아니라, 출구(정답)에서부터 거꾸로 선을 그어가며 미로를 그리는 방법.
- **설명:** “풀어봐” 하고 문제를 주는 게 아니라, 정답을 먼저 고정해 놓고 그 정답이 나올 수밖에 없는 수식과 문제를 AI가 스스로 만들어내게 하는 기법입니다.

- **확증 편향 (Confirmation Bias)**

- **비유:** 첫 단추를 잘못 끼우고 고집부리기.
- **설명:** AI 모델은 자기가 방금 내뱉은 단어에 큰 영향을 받기 때문에, 첫 줄에서 실수를 하면 그 실수를 인정하지 않고 끝까지 엉터리 논리로 오답을 만들어내는 치명적인 고집을 말합니다.

- **XAI (Explainable AI, 설명 가능한 AI)**

- **비유:** 단순 정답지가 아니라 족집게 과외 선생님의 해설지.
- **설명:** “답은 24야”라고 뱉는 블랙박스 AI와 달리, “이래저래 해서 시뮬레이션 방식 대신 공식을 써서 파이썬 코드를 짰고 결과가 이렇게 나왔어”라며 그 과정을 인간의 언어로 설명해 주는 기술입니다.